

Execution plan — market-listing trust layer

Deliverable 3 · Flent FDE take-home · written as I'd put it into Linear

Assumes the approach is approved Monday. One parent, eight sub-issues, ordered.

Two of these already have a working prototype, built during the take-home and deployed: FLT-2 (the layer itself) and FLT-9 (the inspection surface). They are still written as issues because shipping them *inside BOSS*, against live pulls and real reviewers, is the actual work. The prototype proves the pipeline and the refusal; it does not prove they survive contact with a team. Nothing else here can be built without Flent's own data or scraper.

The ordering carries an argument: **ship the refusal before the calibration**, because the fastest way to learn whether this layer is worth building is to watch what an acquisition lead does the first time it says "I don't know."

FLT-1 · Parent — BOSS benchmarks carry a confidence, and can refuse

Type Project · Owner FDE · Cycle 1–3

User outcome

Today the acquisition lead gets a median over 86 scraped rows and no way to argue with it. After v1, every deal shows one of two things:

- a **benchmark** — rate, interval, confidence tier, and a stability verdict saying how hard the call is to overturn; or
- an **explicit refusal** — no number, the reason, and *the single cheapest question that would resolve it*, addressed to a named team.

And in both cases, a listing-by-listing ledger they can open and disagree with. Every one of the 86 rows is present with a status, a rule id, and a sentence in English.

The behaviour change we're buying: "**we don't know yet, go ask X**" becomes a **first-class output** instead of a number nobody trusts but everybody quotes.

v1 includes

- The deterministic pipeline: integrity → cross-post dedup → staleness → scope, with society aliasing ahead of the scope filter
- Benchmark with bootstrap interval and a confidence tier that is **not** a function of sample size
- The robustness battery — the call re-tested under ~20 attacks, reported as the number survived
- Refusal path, with the resolving action and its owner
- Full ledger export; per-listing override by a named human, who outranks every rule in both directions
- One micromarket: Harlur–Sarjapur Road, the five societies in the case packet

v1 explicitly excludes

- **Any model.** Nothing here needs one, and a model makes the exclusions unarguable.
- **A metrics dashboard.** The output is a decision and a question. FLT-9 is an inspection surface with no charts and no aggregate views, and the ordering below is unchanged even though a prototype of it already exists.
- **Photo and description dedup** — needs comp images the scraper does not retain. The packet's four photos are subject-home site-visit stills, not listing photography (FLT-6).
- **Multi-city, and any micromarket outside the packet's five societies.**
- **The rest of the deal.** Capex, tenant revenue and fill-time are larger uncertainties than the benchmark. This layer makes one input honest and says so out loud.
- **Deposit, escalation and rent-free normalisation.** Real, and a different problem.

Release checklist — must be true before this goes in front of Acquisition

1. Every excluded row carries a rule id and a sentence a non-engineer can dispute. No silent deletions; ledger row count equals raw pull row count.
2. The refusal path is exercised on a **real** thin deal, not a constructed one. The fully-furnished configuration of the packet deal (n=4) is the standing test.

3. Every threshold declares EVIDENCED or ASSUMED plus what breaks if it is wrong, and the run prints the split. Currently 4 / 6.
4. An override is attributable to a person and leaves the overridden rule visible in the ledger.
5. Re-running the same pull twice gives the same answer. The bootstrap is seeded.
6. Acquisition lead can read a full output in under five minutes without us in the room.

First metric

Share of refusals closed by evidence within 3 working days, vs. closed by override.

Not accuracy — we cannot measure accuracy until FLT-3 lands. This measures whether the layer changed anyone's behaviour. A trust layer whose refusals are routinely overridden has not reduced risk, it has added a step, and that is worth knowing in week two rather than month six.

Watch alongside it: **count of published benchmarks per week** (does refusal make BOSS useless?) and **median days from refusal to resolution** (is the question actually cheap?).

FLT-2 · Ship the trust layer for one micromarket

Ships first. Prototype already running at flent-trust-layer.vercel.app, scoring the case-packet deal against the 86-row pull. The engine already exists as a script; this is the work of making it something Acquisition opens rather than something an FDE runs.

Outcome. For any deal in Harlur–Sarjapur, the acquisition lead opens BOSS and sees a benchmark with a confidence tier and a stability verdict, or a refusal with a named resolving question. The ledger is one click away and exportable.

Scope boundary. Five societies from the packet. Read-only against the existing scrape — no scraper changes, no new fields. Confidence tiers ship as **stated assumptions, labelled as such in the UI**, because FLT-3 has not returned yet. In: the pipeline, ledger, refusal, override. Out: everything in the parent's exclusion list.

Dependencies. None. This is deliberate — sub-issue 1 must not be blocked on anything, or nothing ships in cycle 1.

Data. Real. The 86-row export from Market operations, live pulls thereafter.

Proof it works. Run against the packet deal and its fully-furnished configuration; the first publishes ₹59,500 / MEDIUM / n=20 with the deviation call refused, the second refuses outright at n=4. Then the real test: three deals scored by the layer and by the current process side by side, reviewed with Acquisition. We are looking for disagreements to be *explicable from the ledger* — not for the two to agree.

One check that is easy to forget: the comp set is selected against the *subject's* furnishing tier, which Supply records by hand and nobody verifies. Furnishing is a ~24% spread in this micromarket, so a mis-recorded tier silently selects the wrong 20 rows. v1 surfaces the tier next to the benchmark so the reviewer can sanity-check it against the site-visit stills already in BOSS.

What we learn. Whether a refusal is treated as useful information or as an obstacle. That answer determines whether FLT-8 (override governance) is a nice-to-have or the most urgent thing in the backlog.

FLT-3 · Calibration join — do asks track achieved rents?

Runs in parallel with FLT-2. No build, no dependency, and it tests the assumption everything else rests on.

Outcome. A measured distribution of `signed rent ÷ benchmark live at signing`, and a yes/no on whether a benchmark built from asking prices is usable as a stand-in for achieved rent.

Why this is second, not first. It was first until I ran `outcomes.csv` (`trust-layer/outcomes_check.py`). Across the ten cases observed to day 90, revenue forecasts land within 2% of plan while fill-time forecasts run 23% long, and payback misses with them. Two of the ten overran by 37 and 39 days — roughly 3.7–3.9 months of payback each — against a median of 6. The rent is not what is hurting these deals; the clock is. That work sits outside this layer, so it is not an issue here, but it should be somebody's issue and it should outrank this one. See "What the outcomes check changes" at the end.

Why it is still worth doing. Listings are asks. A landlord asking ₹70,000 and settling at ₹62,000 leaves only ₹70,000 in the data. We tested the obvious correction — an aspirational ask should sit unsold longer — and got `correlation(days on market, rent) = -0.16`. Wrong sign, negligible. **Nothing in a listings-only dataset separates an aspirational ask from a real one.** Only Flent's own signed rents can.

Scope boundary. A join and a plot. No modelling, no correction factor applied to production in this issue — if a correction is warranted it becomes its own issue with its own review.

Dependencies. Access to 50–100 historical signed rents with signing dates. Owner: Finance / Supply. **This is the long-lead item — request it Monday morning, not in week three.**

Data. Real, and Flent already owns it. If fewer than 50 records are recoverable, that scarcity is itself the finding and FLT-5 gets harder.

Proof it works — with the threshold set in advance.

- **Passes** if the median ratio sits within $\pm 6\%$ and MEDIUM-confidence benchmarks are measurably tighter than LOW ones.
- **Fails** if the ratio is $\sim 8\%$ or worse, or the tiers do not separate.

The 6% is not chosen after the fact: it is the flip point the robustness battery already measured on the packet deal, where the call is stable to 6% ask-inflation and flips at 8%. Pre-registering it is the difference between a test and a rationalisation.

What we learn, and what it unlocks. If it fails, every benchmark in BOSS is biased high and the deviation logic needs a correction factor rather than more filtering — a larger finding than anything in the cleaning pipeline. Either way it unlocks FLT-5, which cannot be built on assumptions.

FLT-4 · Capture the basis at scrape time

Outcome. Every scraped listing carries two new fields: **is the quoted rent all-in or base**, and **is the quoted area carpet, built-up or super built-up** — each with a confidence and the source text it came from.

Why it matters more than any filter. On the packet deal the maintenance convention is worth ₹5,000/month and flips the verdict: against the same ₹59,500 benchmark, the base ask of ₹56,000 reads -5.9% (below market, defensible) and the all-in cost of ₹61,000 reads $+2.5\%$ (above market, negotiate). The information was never scraped. It is **missing, not noisy**, so no cleaning fixes it. Area basis is the same defect class, $\sim 30\%$ wide, and currently corrupts both the dedup key and the sqft-per-bedroom check.

This is the one place AI earns its keep: the basis is usually stated in free text ("rent inclusive of maintenance"), which is a reading problem, not an arithmetic one. **The extractor proposes; it never decides.** Low-confidence extractions leave the field blank and the engine refuses, exactly as it does today.

Scope boundary. Extraction and storage only. In: the two fields, confidence, source snippet, and using them to resolve the fork when present. Out: back-filling history, and any other ambiguous unit.

Dependencies. Market operations owns the scraper. Requires retention of listing description text.

Data. Real, forward-only. Historical rows keep a null basis and keep refusing.

Proof it works. Hand-label 200 listings as ground truth. Ship only at $\geq 95\%$ **precision on the labelled set** — precision, not recall, because a wrong basis silently produces a confident wrong verdict, whereas a blank one produces a refusal we already handle correctly. Then: the packet deal, currently `INSUFFICIENT` / `UNRESOLVED`, resolves to a determinate call once basis is known for the comps.

What it unlocks. Converts a whole category of refusal into a lookup, which should show up directly in the parent's first metric.

FLT-5 · Replace the `min_n` cliff with a graded confidence floor

Outcome. Confidence degrades continuously instead of stepping off a cliff at an arbitrary count, and the floor is derived from measurement rather than judgment.

The defect. `min_n_low = 6` is the least defended number in the engine — chosen, not measured — and it is a step function applied to a dirty input. Two fabricated rows turn the fully-furnished refusal ($n=4$) into a published ₹68,750 at LOW confidence. The robustness battery surfaces that as `FRAGILE`, which is a mitigation, not a fix.

Scope boundary. In: replacing the two `min_n` thresholds and re-deriving the tier boundaries from FLT-3's calibration data. Out: any other threshold. The other four `ASSUMED` entries stay assumed and stay labelled.

Dependencies. **Hard block on FLT-3.** A confidence tier is a claim about future accuracy. It cannot be validated from listings, no matter how many we clean — that is the honest boundary of what this layer can prove about itself, and building this before FLT-3 returns would just relocate a guess.

Data. Real: FLT-3's signed rents. The two-row cliff demo stays as a regression test and stays **labelled as fabricated**.

Proof it works. The cliff test no longer flips: adding two rows to the $n=4$ case must not produce a publishable benchmark. And on the historical set, realised error inside each tier is ordered as the tier names claim — HIGH tighter than MEDIUM tighter than LOW. If that ordering does not hold, the tiers are decoration and we say so.

FLT-6 · Content-based cross-post detection

Outcome. Duplicate detection keyed on what the listing is — photo hashes and description similarity — rather than on a date-and-area tolerance that is evadable by construction.

The defect. The current rule requires society, area, BHK, rent within 2% and posted dates within 2 days. It catches 12 of 12 cross-posts here, but a tolerance is always evadable beyond its width: a 3-day shift leaks, and widening costs precision (14 days gives 4 false merges for zero extra recall). Portals are scraped on different days *by definition*, so this rule is weaker in production than it looks on the packet.

Why it is not higher in the order. Under-merging is the *recoverable* error — it shrinks a comp set visibly. Over-merging destroys evidence invisibly. The current rule errs toward under-merging, so the defect it leaves is the one we can live with while FLT-3 and FLT-4 run. It is also bounded today: attack A1 re-tests the call assuming dedup caught nothing at all.

Scope boundary. In: perceptual hashing of listing images, description similarity, and merging the two signals with the existing structural key. Out: cross-*society* matching, and any use of these signals outside dedup.

Dependencies. Requires the scraper to retain images — same conversation as FLT-4, so raise both with Market operations at once. Not blocked on FLT-3.

Data. Real going forward. The packet's only images are four site-visit stills of the *subject* home; `listings.csv` carries a `photo_count` field but no marketplace images, which the packet states explicitly. So **nothing in this issue can be validated on the case packet** and we should not pretend otherwise.

Proof it works. Hand-label cross-post clusters in one week of live pulls. Target: recall above the current structural rule at no loss of precision, measured on the date-shifted pull that breaks the current rule. Ship behind a flag, compare merge decisions against the structural rule for two weeks, and review every disagreement by hand before switching over.

FLT-7 · Detect bait by the gap between the posted rent and the quoted one

Outcome. A bait score on every listing, built from signals the current pull does not carry: what the landlord quotes when somebody actually calls, how often a listing is reposted without ever leaving the market, and how many listings one poster runs below their own median. Bait stops being a defect we route around and becomes a field we can filter on.

The defect. Four rows at ₹51,000–₹52,500 survived every filter in the pipeline and pulled the median from ₹59,500 to ₹58,500. Bait posted to harvest leads is *plausibly* cheap, not absurdly cheap — it sits **inside** the distribution, so no outlier test can reach it. The ₹12,000 and ₹1,85,000 rows the engine does catch were free wins and are not evidence the test works. This is the one high-severity defect in `FAILURES.md` with no other issue against it: FLT-4 and FLT-6 make listings more comparable, and neither makes a cheap listing more honest.

Why here in the order. It is bounded today and the bound is reassuring: attack A3.k re-tests the call with the *k* cheapest listings removed, for *k* up to 15% of the sample, and the packet deal survives. So this is not urgent in the way FLT-3 is. It sits beside FLT-6 because both ask "what is this listing really?" rather than "how do we compare it", and because the poster-identity signal falls out of the same scrape change.

The signal that actually works, and why it is not in the data. A bait listing is *defined* by the price changing when you contact it. That is not a property of the listing; it is a property of the conversation, and Flent has those conversations every week. Supply already calls landlords. Capturing the quoted rent on that call — one field — turns an undetectable defect into a labelled dataset. This is the same shape as the maintenance question the layer already asks: the missing fact is one phone call away, and nobody is writing it down.

Scope boundary. In: capturing quoted-versus-posted rent on Supply calls, poster identity clustering, repost churn, and a score combining them. Out: acting on the score automatically. A bait score demotes a listing's weight or flags it for review; it does not silently delete rows, because a wrongly flagged cheap listing is exactly how a benchmark gets biased upward — and a trust layer that quietly removes the low end is worse than one that admits it cannot tell.

Dependencies. A one-field process change with Supply, and poster identity retained at scrape time — the same conversation as FLT-4 and FLT-6, so raise all three with Market operations at once. Not blocked on FLT-3.

Data. Real going forward, and **nothing here can be validated on the case packet**. `listings.csv` has no poster identifier beyond `poster_type`, no repost history, and no call outcomes. The four suspected bait rows are suspected on

price alone; the packet cannot confirm they are bait. Saying so matters more than the feature does — the whole argument of this layer is that an unverifiable claim gets labelled, not shipped.

Proof it works — with the threshold set in advance.

- **Passes** if listings flagged by the score show a materially larger posted-to-quoted gap than unflagged ones on 50+ calls, and removing them moves a benchmark by less than the A3.k bound already assumes. The score has to beat the crude defence we already have.
- **Fails** if the gap does not separate, or if the flags concentrate on one portal — which would mean we built a portal detector and called it a bait detector.

What we learn either way. If it fails, the honest answer is that bait is not separable from a genuinely cheap listing without transaction data, and A3.k stays the defence: the call is reported as surviving the removal of the cheapest 15%, and we say that out loud rather than implying the low end is clean.

FLT-8 · Override governance — identity and an append-only log

Outcome. An override is an attributable, reviewable act rather than a free-text name somebody types.

The defect, stated plainly. `overrides.csv` lets anyone force any listing in or out, signed with a name they supply themselves. Someone motivated to close a deal can force in exactly the comps that flatter the ask. **The feature built for trust is the cheapest way to launder a number.** Nothing in a local script fixes this — it needs identity, an append-only log, and a reviewer who is not the deal owner.

Scope boundary. In: authenticated identity on every override, an append-only audit log, and a rule that the deal owner cannot be the sole approver of overrides on their own deal. Out: a full approvals workflow, and any attempt to restrict *which* rules can be overridden. A named human still outranks every rule in both directions — that is the point of the feature.

Dependencies. BOSS auth. None on FLT-3 or FLT-4.

Priority is conditional, and FLT-2's metric sets it. If overrides are rare, this can wait. If refusals are being closed by override rather than by evidence, this becomes the most urgent issue in the backlog and jumps the queue.

Data. Real usage from FLT-2.

Proof it works. Every override in the log is attributable to an authenticated person with a reason and a timestamp; the log cannot be edited after the fact; and the weekly override report is reviewed by someone other than the deal owner.

FLT-9 · Ledger inspection UI

Prototype already built, ahead of its place in this order. The thesis is *a decision, not a dashboard*, and that still governs what it contains: no charts, no aggregate market views, every row carrying the rule and sentence that excluded it. What remains is the part the prototype cannot do — wiring it to live pulls and to the audited override path from FLT-8, so a reviewer's disagreement is recorded rather than kept in their browser.

Outcome. Acquisition and Supply can filter the ledger, see why any listing was excluded, and file an override in place — with the audit trail from FLT-8 behind it.

Scope boundary. In: filter, sort, per-row rule and reason, one-click override, export. Out: charts, trend lines, aggregate market views. Anything that invites reading a number off a screen without its ledger reintroduces the exact failure this project exists to fix — *a bad market rate looks just as precise on a dashboard as a good one*.

Dependencies. FLT-2 for the data, FLT-8 for the override path. The prototype currently stores a reviewer's marks in their own browser, which is exactly the gap FLT-8 closes.

Data. Real.

Proof it works. A Supply reviewer who has not seen the tool finds why a specific listing was excluded, and files a justified override, in under two minutes without help.

The shape of the sequence

Cycle 1 — FLT-2 ships the layer; FLT-3 requests the signed rents on day one and runs the join. One delivers a working thing, the other delivers a finding. Neither blocks the other.

Cycle 2 — FLT-4, FLT-6 and FLT-7 go to Market operations together, because all three are scrape-time changes and there is one conversation to have, not three. FLT-7 also needs a one-field change from Supply, which is a separate ask on the same day. FLT-5 starts the moment FLT-3 returns.

Cycle 3 — FLT-8 at whatever priority FLT-2's override rate has earned it, then FLT-9.

What the outcomes check changes

`trust-layer/outcomes_check.py` reads the packet's own acquisition history. It says the larger measurement error at Flent is fill time, not market rate — 23% long versus 2%, with a tail of two deals in ten running five weeks over. Ten observed cases and $p = 0.11$ is not significance, and the excluded still-filling case biases it optimistic, so treat it as evidence rather than proof.

I have not turned it into an issue in this plan, because correcting the fill-time forecast is BOSS's underwriting, not this layer, and the brief asks for one problem attacked properly rather than a second one bolted on. But the ordering matters more than the boundary: **if I could add one issue outside Problem 1, it would be the fill-time calibration, and it would outrank FLT-3.** The tail shape decides the fix — a systematic bias takes a multiplier, a two-in-ten blowout takes early detection, and those are different pieces of work.

The through-line: **two of the three fixes that most improve this layer are scraping changes, not modelling changes**, and the third is a join against data Flent already owns. The highest-value work on Problem 1 is capturing more about each listing and learning what our own signed deals say — not filtering the existing fields harder.